



Desafios - Aula 2

🕒 Criado em	7 de setembro de 2026 09:47
🏷 Tags	

Um sistema distribuído não é simplesmente "colocar vários computadores para trabalharem juntos". Quando distribuímos um sistema, surgem problemas que praticamente não existem — ou são muito mais simples — quando tudo está em uma única máquina.

Os principais desafios estudados são:

1. **Heterogeneidade**
2. **Abertura**
3. **Segurança**
4. **Escalabilidade**
5. **Tratamento de falhas**
6. **Concorrência**
7. **Transparência**

1. Heterogeneidade

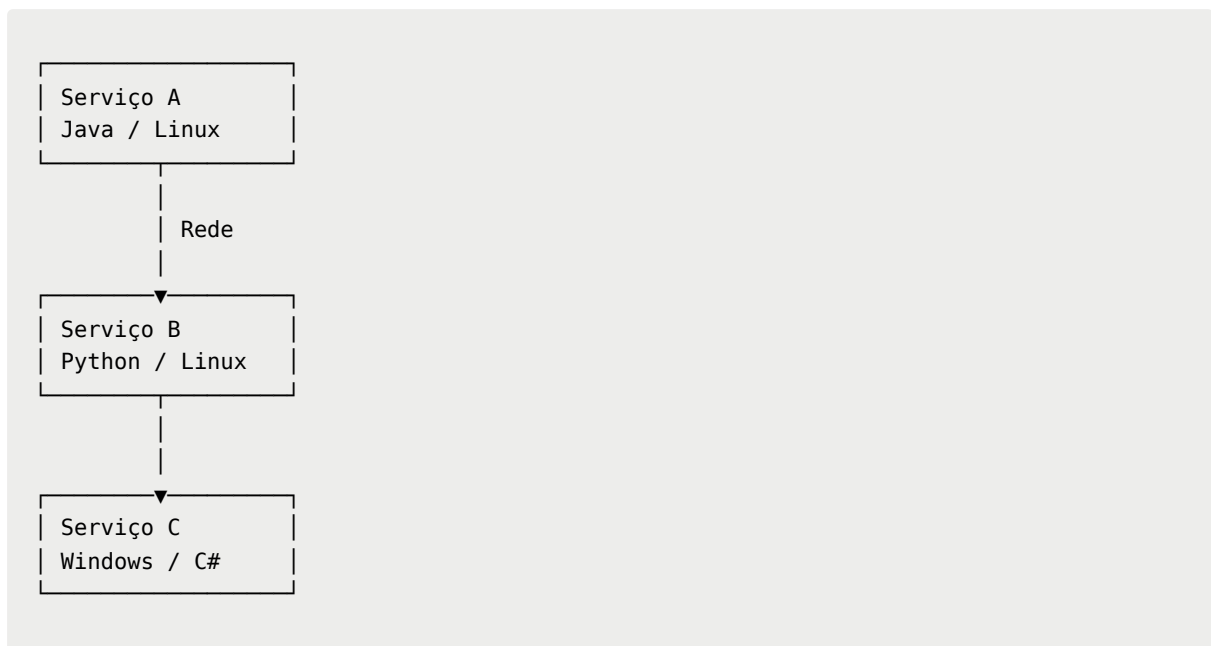
O que significa?

Heterogeneidade significa que os diferentes componentes de um sistema distribuído podem possuir características diferentes.

Podemos ter diferenças em:

- hardware;
- sistemas operacionais;
- linguagens de programação;
- arquiteturas de processador;
- redes;
- protocolos;
- bancos de dados;
- tecnologias utilizadas.

Imagine:



Mesmo sendo tecnologias diferentes, elas precisam conseguir **se comunicar e cooperar**.

Problema

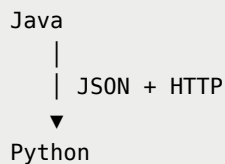
Como fazer componentes tão diferentes trabalharem juntos?

É por isso que existem padrões e tecnologias de comunicação, como:

- HTTP;

- JSON;
- REST;
- TCP/IP;
- gRPC;
- protocolos de mensageria.

Por exemplo:



O Java não precisa entender como o Python foi implementado internamente. Eles concordam com um **formato e protocolo de comunicação**.

No mercado

Isso é muito comum.

Uma empresa pode ter:

```
Frontend → Angular
Backend → Java
IA → Python
Banco → PostgreSQL
Mensageria → Kafka
Infraestrutura → Linux
```

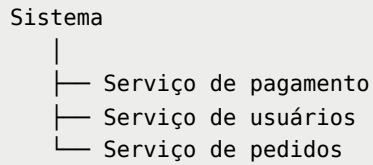
A heterogeneidade é praticamente inevitável em sistemas grandes.

2. Abertura

Um sistema distribuído é considerado **aberto** quando seus componentes podem ser desenvolvidos, substituídos ou integrados utilizando interfaces e padrões bem definidos.

A ideia é evitar que o sistema fique completamente preso a uma implementação específica.

Imagine:



Se o serviço de pagamento possui uma interface bem definida:

POST /pagamento

podemos trocar sua implementação sem necessariamente modificar todo o restante do sistema.

O objetivo

Facilitar:

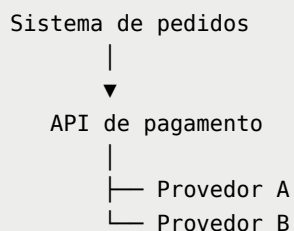
- integração;
- extensão;
- substituição de componentes;
- interoperabilidade;
- evolução do sistema.



No mercado

Isso aparece bastante em arquiteturas baseadas em **APIs e microsserviços**.

Por exemplo:

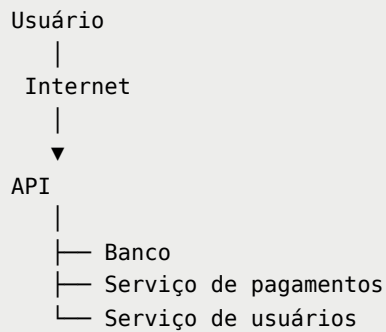


O restante do sistema conversa com uma interface, enquanto a implementação pode mudar.

3. Segurança

Quando os componentes estão conectados por uma rede, existe uma superfície maior para ataques.

Imagine:



Existem várias comunicações acontecendo.

Precisamos proteger:

- dados;
- identidade dos usuários;
- autenticação;
- autorização;
- comunicação;
- serviços;
- armazenamento.

Três conceitos importantes

Confidencialidade

Somente pessoas autorizadas devem conseguir acessar determinadas informações.

Integridade

Os dados não devem ser alterados de maneira indevida.

Disponibilidade

O serviço precisa continuar acessível para usuários autorizados.

Esses três conceitos aparecem frequentemente associados à segurança da informação.



No mercado

Você vai encontrar mecanismos como:

- HTTPS/TLS;
- autenticação;
- autorização;
- tokens;
- OAuth 2.0;
- JWT;
- controle de acesso;
- criptografia;
- firewalls;
- gerenciamento de secrets.

E isso é especialmente importante quando vários serviços estão conversando entre si.

4. Escalabilidade

Já vimos que um dos principais motivos para distribuir um sistema é permitir que ele cresça.

Mas **escalar não significa simplesmente adicionar máquinas**.

Existem diferentes dimensões de escalabilidade.

Escalabilidade de tamanho

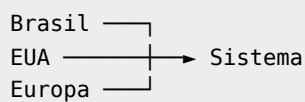
O número de usuários, dispositivos ou dados aumenta.

```
1.000 usuários
  ↓
100.000 usuários
  ↓
10.000.000 usuários
```

O sistema precisa continuar funcionando.

Escalabilidade geográfica

Os usuários podem estar espalhados por diferentes regiões.



Escalabilidade administrativa

À medida que o sistema cresce, administrar seus componentes também fica mais difícil.

Não basta perguntar:

| "Temos máquinas suficientes?"

Também precisamos perguntar:

| "Conseguimos administrar tudo isso?"



No mercado

É aqui que aparecem conceitos como:

- load balancer;
- replicação;
- cache;
- filas;
- microsserviços;
- containers;
- Kubernetes;

- cloud computing;
- bancos distribuídos.

5. Tratamento de falhas

Esse é um dos **problemas mais importantes de Sistemas Distribuídos**.

Em um sistema distribuído, componentes podem falhar independentemente.

Servidor A ✓

Servidor B ✗

Servidor C ✓

O sistema precisa ser capaz de perceber e lidar com situações como:

- servidor indisponível;
- conexão perdida;
- mensagem não recebida;
- resposta atrasada;
- banco indisponível;
- serviço sobrecarregado.

O problema da rede

Imagine:

A → B

A envia uma mensagem.

B não responde.

O que aconteceu?

Possibilidades:

1. B recebeu e está processando;
2. B recebeu, mas demorou para responder;
3. a mensagem não chegou;
4. a resposta foi perdida;
5. B caiu;
6. a rede está com problemas.

A máquina A **não consegue simplesmente assumir** qual dessas situações aconteceu.

Essa incerteza é uma das razões pelas quais sistemas distribuídos são difíceis.



No mercado

Isso aparece em mecanismos como:

- timeout;
- retry;
- circuit breaker;
- fallback;
- replicação;
- health checks;
- failover.

Mas cuidado:

Retry não é simplesmente "tentar infinitamente".

Se um serviço está sobrecarregado e todos os clientes começam a repetir suas requisições, podemos piorar ainda mais o problema.

Esse tipo de raciocínio é muito importante em arquitetura de sistemas.

6. Concorrência

Em sistemas distribuídos, vários processos podem executar simultaneamente.

Imagine dois usuários comprando o último produto disponível:

```
Estoque = 1  
  
Usuário A → comprar  
Usuário B → comprar
```

As duas operações podem acontecer praticamente ao mesmo tempo.

Isso pode gerar problemas de **consistência**.

Por exemplo:

```
Estoque = 1  
  
A lê → 1  
B lê → 1  
  
A compra → 0  
B compra → 0
```

O sistema acabou permitindo duas compras quando existia apenas um produto.

Isso é um exemplo simplificado de um problema de concorrência.

O desafio

Precisamos controlar como operações simultâneas interagem.

Isso envolve conceitos como:

- sincronização;
- exclusão mútua;
- locks;
- transações;
- consistência;
- condições de corrida (*race conditions*).



No mercado

Esse problema aparece constantemente em:

- bancos;

- sistemas de pagamento;
- reservas;
- estoques;
- processamento de pedidos;
- bancos de dados;
- sistemas de filas.

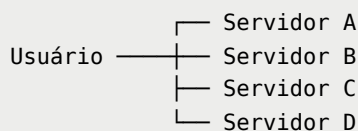
7. Transparência

Transparência significa que o sistema tenta **esconder do usuário e/ou desenvolvedor a complexidade causada pela distribuição**.

Imagine que você acessa:

```
https://meusistema.com
```

Você não sabe necessariamente:



E provavelmente nem deveria precisar saber.

Você simplesmente utiliza:

```
meusistema.com
```

O sistema pode decidir internamente para onde sua requisição será enviada.

Existem diferentes formas de transparência

Dependendo da abordagem estudada pelo professor, podem aparecer conceitos como:

- **transparência de acesso** — esconder diferenças na forma de acessar recursos;

- **transparência de localização** — esconder onde o recurso está;
- **transparência de migração** — esconder que um recurso mudou de localização;
- **transparência de replicação** — esconder que existem várias cópias;
- **transparência de concorrência** — esconder que vários usuários compartilham recursos;
- **transparência de falhas** — tentar esconder determinadas falhas do usuário.

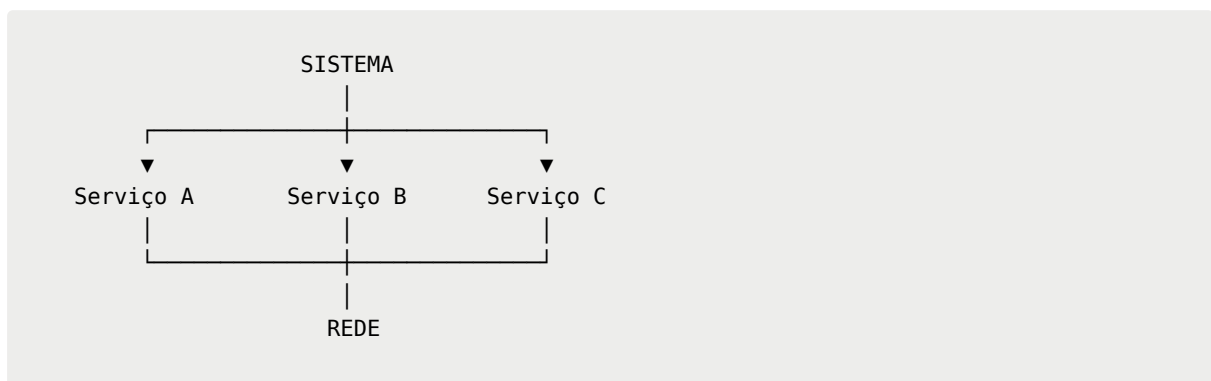
A ideia geral é:

O sistema é distribuído por dentro, mas tenta apresentar uma experiência mais simples por fora.

Como os desafios se relacionam?

O mais importante é não decorar esses sete conceitos como uma lista isolada. Eles estão conectados.

Imagine uma aplicação crescendo:



Agora começam os problemas:

Heterogeneidade

| E se A for Java e B for Python?

Abertura

| Como permitir que novos serviços sejam integrados?

Segurança

| Como impedir que qualquer pessoa acesse os serviços?

Escalabilidade

| E se tivermos milhões de requisições?

Falhas

| E se B cair?

Concorrência

| E se A e C modificarem o mesmo recurso simultaneamente?

Transparência

| Como esconder toda essa complexidade do usuário?

Perceba que **distribuir o sistema resolve alguns problemas, mas cria vários outros.**

Essa é uma das ideias mais importantes da Unidade I.



Resumo para memorizar

Desafio	Pergunta principal
Heterogeneidade	Como componentes diferentes trabalham juntos?
Abertura	Como integrar, substituir e expandir componentes?
Segurança	Como proteger dados e comunicação?
Escalabilidade	Como o sistema continua funcionando ao crescer?
Tratamento de falhas	Como continuar funcionando quando componentes falham?
Concorrência	Como lidar com várias operações acontecendo simultaneamente?
Transparência	Como esconder a complexidade da distribuição?



Para estudo ativo

Tente explicar, **sem olhar o material**, esta situação:

Uma empresa possui uma aplicação com Angular, vários serviços Spring Boot, um banco de dados e um sistema de pagamentos externo. O sistema cresceu de 1.000 para 1 milhão de usuários.

Agora identifique **onde cada um dos sete desafios aparece**.

Se você conseguir olhar para uma arquitetura real e falar *"aqui temos um problema de escalabilidade; aqui de falha; aqui de concorrência..."*, você está começando a **entender Sistemas Distribuídos de verdade**, em vez de só memorizar a lista do professor.